CSE 574A: Introduction to Machine Learning

Project 6

Curve Fitting Using a Neural Network
with the Backpropagation Algorithm

Submitted By: Viral Koshti
Student ID: 50761354

---

1 Objective

This project aims to fit a non-linear curve by training a feedforward neural network with one input neuron using the backpropagation (BP) algorithm. I trained the network using 300 randomly generated data points with a small amount of noise.

The main task was to minimize the mean squared error (MSE) using online learning, where the network weights are updated after each training example. I also adjusted the learning rate during training to prevent the error from increasing when the weight updates were too large.

2 Dataset Generation

The training dataset contains 300 input values sampled uniformly from the interval $[0, 1]$:

$$x_i \sim U(0, 1), \qquad i = 1, \ldots, 300. \tag{1}$$

For every input, random noise was generated from:

$$\nu_i \sim U(-0.1, 0.1). \tag{2}$$

The corresponding target output was calculated using:

$$d_i = \sin(20x_i) + 3x_i + \nu_i. \tag{3}$$

The sinusoidal term produces the oscillations in the target curve, while the linear term $3x_i$ creates an increasing trend. The added noise makes the generated points slightly different from the original function.

I used NumPy's random number generator with a fixed seed of 42 to make the experiment reproducible.

Figure 1 shows the generated data.

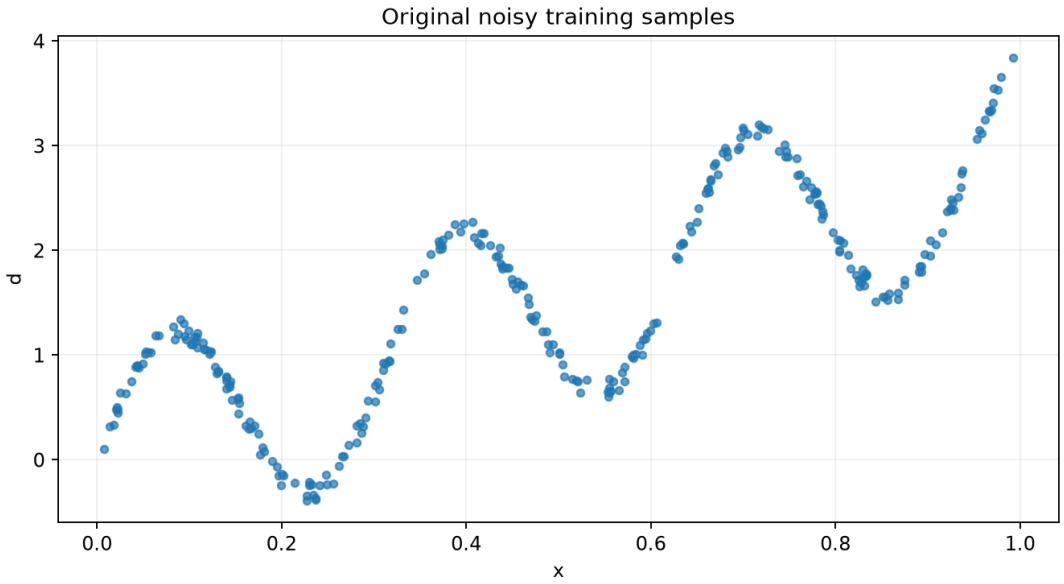


Figure 1: The 300 randomly generated training samples with uniformly distributed noise.

3 Neural Network Architecture

I used a three-layer feedforward neural network with one input neuron, 24 hidden neurons, and one output neuron. The network architecture is therefore:

$$1 \times 24 \times 1. \tag{4}$$

The hidden neurons use the hyperbolic tangent activation function:

$$\phi(v) = \tanh(v). \tag{5}$$

The output neuron uses a linear activation function:

$$\phi(v) = v. \tag{6}$$

For an input x, the output of hidden neuron j is:

$$h_j = \tanh(w_{1j}x + b_{1j}), \qquad j = 1, \ldots, 24. \tag{7}$$

The final network prediction is:

$$f(x, \mathbf{w}) = \sum_{j=1}^{24} w_{2j} h_j + b_2. \tag{8}$$

Here, w_{1j} and b_{1j} represent the input weight and bias of hidden neuron j. The values w_{2j} represent the hidden-to-output weights, and b_2 is the output bias.

The total number of trainable parameters is:

$$24 + 24 + 24 + 1 = \boxed{73}. \tag{9}$$

All 73 parameters are updated during backpropagation.

I initialized the hidden neurons with different weights and biases so that they could respond to different parts of the input range. The output weights were initialized with small random values. All weights and biases were then updated during training.

4 Backpropagation and Online Learning

The objective was to minimize the mean squared error between the target values and network predictions:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}\left(d_i - f(x_i, \mathbf{w})\right)^2 . \tag{10}$$

For each training example, I used the following sample loss to calculate the gradients:

$$E_i = \frac{1}{2}(d_i - y_i)^2, \tag{11}$$

where y_i is the predicted output of the network.

The output-layer error term is:

$$\delta_o = y_i - d_i. \tag{12}$$

Since the output activation is linear, its derivative is equal to one.

For each hidden neuron, the error term is:

$$\delta_j = \delta_o w_{2j}(1 - h_j^2). \tag{13}$$

The expression $(1 - h_j^2)$ is the derivative of the tanh activation function.

The output-layer weights and bias are updated as:

$$w_{2j} \leftarrow w_{2j} - \eta\delta_o h_j, \tag{14}$$
$$b_2 \leftarrow b_2 - \eta\delta_o. \tag{15}$$

The hidden-layer weights and biases are updated as:

$$w_{1j} \leftarrow w_{1j} - \eta\delta_j x_i, \tag{16}$$
$$b_{1j} \leftarrow b_{1j} - \eta\delta_j. \tag{17}$$

Here, η is the learning rate.

I used online learning, meaning the weights are updated immediately after processing each individual training sample. The 300 samples were shuffled at the beginning of each epoch.

5 Learning Rate Adjustment

The initial learning rate was set to:

$$\eta_0 = 0.008. \tag{18}$$

A relatively large learning rate can cause the network to overshoot a better solution, which may increase the MSE instead of decreasing it.

To handle this, I checked the MSE after every training epoch and compared it with the previous accepted MSE.

If the MSE increased beyond a small numerical tolerance, I restored the weights from the start of that epoch and reduced the learning rate using:

$$\boxed{\eta_{\text{new}} = 0.9\eta_{\text{old}}}. \tag{19}$$

The next epoch was then attempted with the smaller learning rate.

I also set a minimum learning rate of:

$$\eta_{\min} = 0.00004. \tag{20}$$

The learning rate was reduced 42 times during training. Its final value was approximately:

$$\eta_{\text{final}} = 0.00009578. \tag{21}$$

This approach allowed the training process to retain weight updates that reduced the error while rejecting epochs that increased it.

6 Experimental Results

The network was trained for 2200 epochs using the generated dataset.

Table 1 summarizes the training configuration and results.

Table 1: Training configuration and results

Parameter	**Value**
Number of samples	300
Hidden neurons	24
Trainable parameters	73
Training method	Online BP
Random seed	42
Number of epochs	2200
Initial learning rate	0.008
Final learning rate	0.00009578
Learning rate reductions	42
Initial MSE	0.664258
Final MSE	0.003511

6.1 Mean Squared Error During Training

Figure 2 shows the MSE recorded after each epoch.

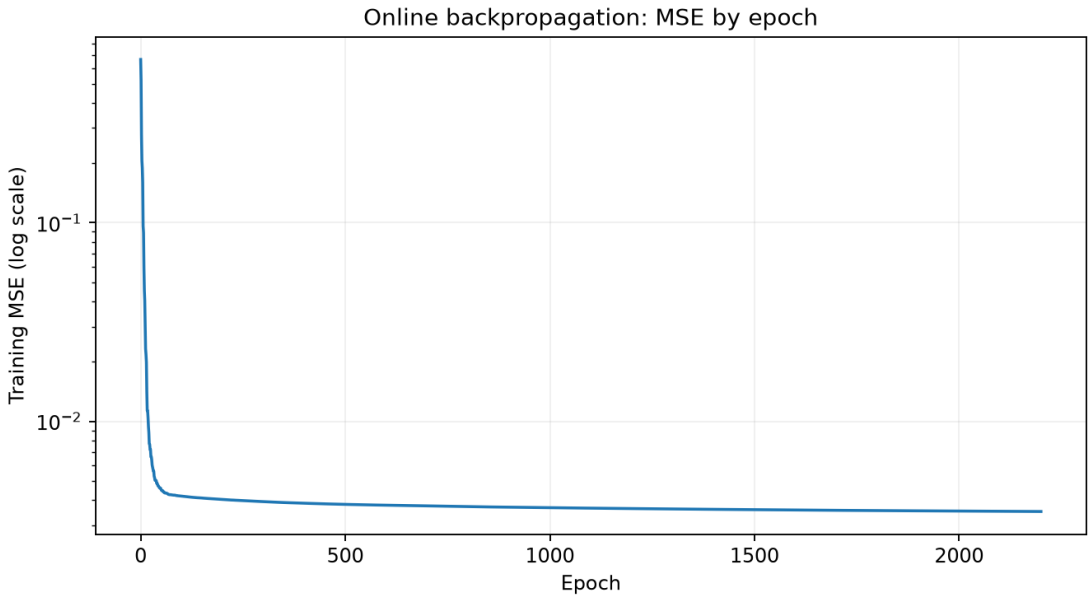


Figure 2: Mean squared error versus training epoch using online backpropagation (logarithmic y-axis).

The initial MSE was approximately 0.664258. As training continued, the error decreased, reaching approximately 0.003511 after 2200 epochs.

The adaptive learning rate helped control the training process when an epoch produced a higher MSE. Since the previous weights were restored after a rejected epoch, the recorded accepted MSE did not increase.

6.2 Final Curve Fitting

After training, the final network weights were used to generate predictions for 600 equally spaced input values between 0 and 1.

Figure 3 shows the fitted neural network curve together with the noisy training samples and the original noise-free function.

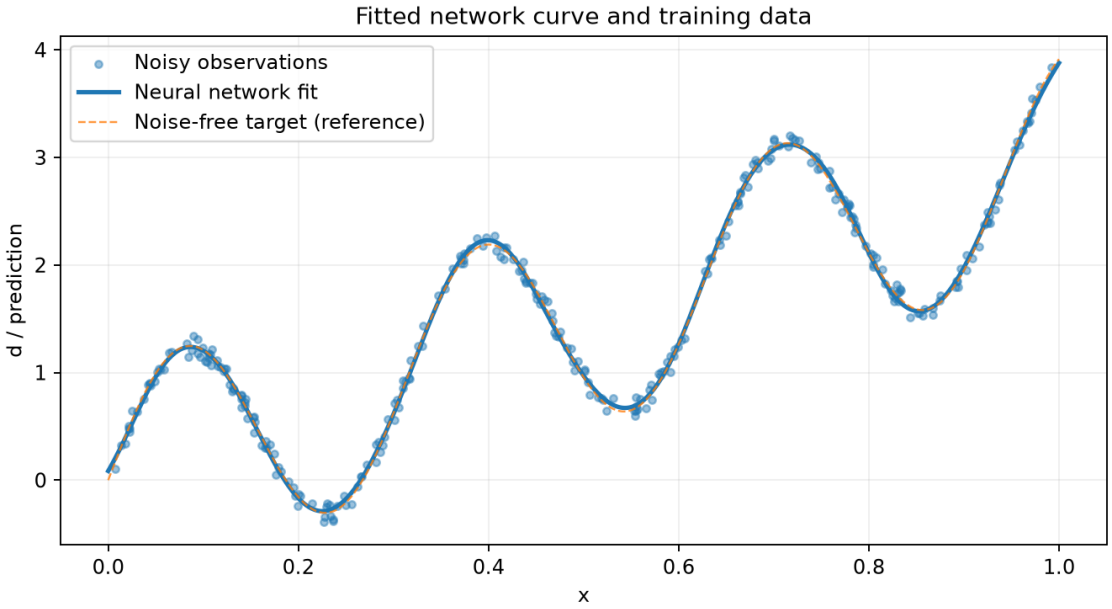


Figure 3: Final neural network fit compared with the noisy training samples and the original noise-free target function.

The fitted curve follows the oscillating pattern of the generated data while also capturing the overall increasing trend.

The prediction is smooth even though the training samples contain random noise. The low final MSE indicates that the network learned the training data closely.

7 Conclusion

For this project, I used a $1 \times 24 \times 1$ neural network and trained it using online backpropagation. The training MSE decreased from 0.664258 to 0.003511 after 2200 epochs.

During training, I noticed that the learning rate affected how the error decreased. When an epoch produced a higher MSE, I restored the previous weights and reduced the learning rate. This helped avoid updates that made the error worse.

The final fitted curve follows the original function closely, even though the training data contains some random noise. From this experiment, I understood how backpropagation and learning rate adjustments help a neural network learn a nonlinear function.